Assignment 1 - Git & GitHub Fundamentals

Report Template

- **Full Name:** Yahya HZA
- **Student ID:** 120379
- **Course Code & Section:** CSC 3324 02, Group D
- **GitHub Username:** HDblinky

1. Repository Setup

I created the folder git-session1-HDblinky, copied the starter files into it, and ran "git init -b main" to initialize the repository with main as the default branch. My .gitignore ignores local.env and all .log files with "*.log", and the line "!important.log" makes an exception so that important.log stays visible to Git. I checked this with "git status --ignored" and "git ls-files": important.log is tracked, while local.env, app.log and debug.log are not.

2. README & Commit History

My README contains my name, GitHub username, a short project description, a list of technologies, one code block, one link, and the project banner image from the assets folder. I made each commit about one logical change: .gitignore, important.log, index.html, style.css, app.js, notes.txt, and the README with the banner. This gave me more than 5 commits across more than 3 files, each with a message describing what was added.

3. Selective Staging

I edited style.css and app.js at the same time, so "git status" showed both as modified. I ran "git add style.css" and committed only that file. "git status" then showed app.js still modified but unstaged. I used "git diff" to see that unstaged change, then "git add app.js" and "git diff --staged" to see the staged one, and committed it afterwards.

4. Branch & Merge

I created the branch about-feature with "git switch -c about-feature" and committed the About page there. Then I went back to main with "git switch main" and ran "git merge --no-ff about-feature", which created a merge commit so the branch is visible in "git log --oneline --graph --all".

5. Remote Changes

A commit saves a snapshot of my work in my local repository only, while a push uploads my local commits to GitHub. A fetch downloads new commits from GitHub into my local copy of the remote branch (origin/main) without changing my working files, so I can check what changed first. A pull is a fetch followed by a merge into my current branch, so my files are updated. I edited README.md on GitHub, ran "git fetch" and "git status" to see that the remote had a commit I didn't, and then ran "git pull" to bring it in. [If your fetch showed "diverged" because you had pushed local commits first, say so here in one sentence.]

6. Fair Use of AI

I used Claude to help me understand and troubleshoot the Git workflow. I asked it to explain the assignment steps, how the ! rule in .gitignore works, the difference between git fetch and git pull, and what to do about errors such as "pathspec did not match any files", "remote origin already exists", and a rejected push. I also asked how to leave the Vim editor during the merge commit. I ran every command myself and checked the results with git status, git log --oneline --graph --all, git ls-files and git remote -v. Some of the early advice didn't fit my situation. For example, my first commands ran in the wrong repository because the project folder had no .git of its own, so I had to run git init in the right folder. I also had to correct my plan after checking my history showed the About branch had not been merged yet. I didn't just copy commands: I checked each result and made sure I understood what each command does.

7. Sources & Reflection

Sources: Pro Git Book (https://git-scm.com/book/en/v2), the Git documentation (https://git-scm.com/docs), GitHub Docs, the Session 1 course material, and Claude for explanations and troubleshooting.

The hardest concept for me was understanding the difference between my working folder, my local repository, and the remote on GitHub, especially fetch vs pull and why a push gets rejected when the remote has new commits. If Git behaves unexpectedly, I would first run "git status" to see my branch and file state, then "git rev-parse --show-toplevel" to confirm I'm in the right repository, and "git remote -v" and "git log --oneline --graph --all" to check where my history stands.